

Object-Oriented Programming UML and Object Relationships

100年度教育部資訊軟體人才培育推動中心
軟體工程推廣分項計畫

編輯委員 國立台北科技大學 陳偉凱教授

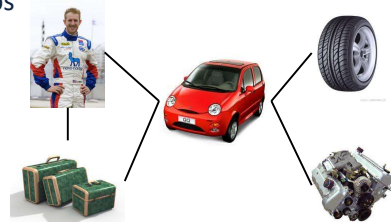


Software Development and Test Laboratory • Department of Computer Science and Information Engineering • National Taipei University of Technology

1

Object relationships

- An object-oriented program may be viewed as a collection of **interacting objects**
- UML** can be used to express object relationships



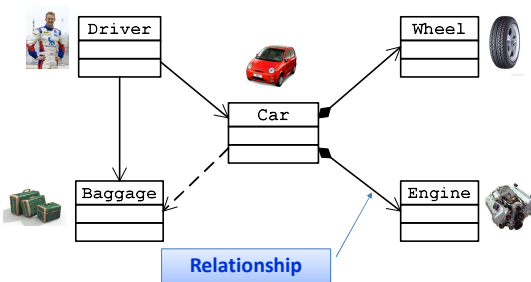
Object-Oriented Programming UML and Object Relationships

2 / 36

Software Development and Test Laboratory • Department of Computer Science and Information Engineering • National Taipei University of Technology

2

An example UML class diagram



Object-Oriented Programming UML and Object Relationships

3 / 36

Software Development and Test Laboratory • Department of Computer Science and Information Engineering • National Taipei University of Technology

3

UML: Unified Modeling Language

- Successor to OOAD methods
 - late 1980s and early 1990s
- Unifies
 - Jacobson & OMT (Booch & Rumbaugh)
- Graphical notation used to express designs
 - Use cases
 - Class diagrams
 - Interaction diagrams (Sequence & communication diagrams)
 - Package diagrams
 - State diagrams
 - Activity diagrams
 - Deployment diagrams

GoF Book



Object-Oriented Programming UML and Object Relationships

4 / 36

Software Development and Test Laboratory • Department of Computer Science and Information Engineering • National Taipei University of Technology

4

UML class diagrams

- Can be used for different perspectives
 - Conceptual**
 - Describe things in a situation of the real world or domain of interest
 - E.g., domain model
 - Implementation**
 - Describe software implementations in a particular technology (e.g., Java)
 - E.g. design class diagram

Covered in this lecture



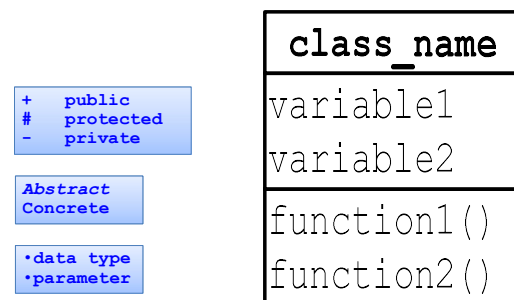
Object-Oriented Programming UML and Object Relationships

5 / 36

Software Development and Test Laboratory • Department of Computer Science and Information Engineering • National Taipei University of Technology

5

UML: Class



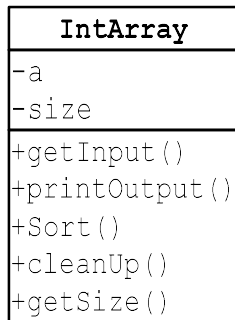
Object-Oriented Programming UML and Object Relationships

6 / 36

Software Development and Test Laboratory • Department of Computer Science and Information Engineering • National Taipei University of Technology

6

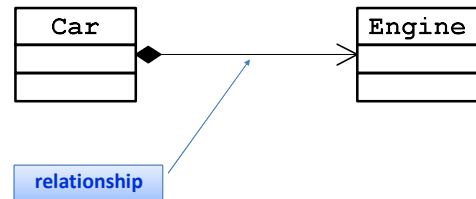
Example: Class



7

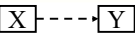
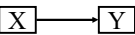
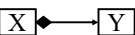
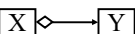
Example: Relationship

- An object is related to some other objects



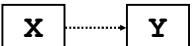
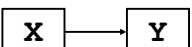
8

UML: class relationship

- Dependency  (uses a)
- Association  (knows a)
- Composition  (has a)
- Aggregation  (has a)
- Inheritance  (is a)
- Implementation  (can do)

9

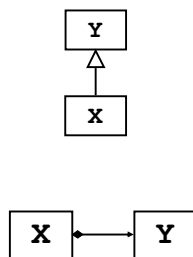
"Uses a" ⇔ "Knows a" relationship

- "Uses a"
 - Dependency 
 - X issues a function call to a member function of Y
 - Short term relationship
- "Knows a"
 - Association 
 - X is aware of Y; X contains a pointer or reference to Y
 - X can issue a function call to a member function of Y
 - Long term relationship

10

"Is a" ⇔ "Has a" relationship

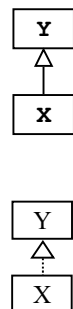
- "Is a" relationships
 - Inheritance
 - X class is derived from Y
- "Has a" relationships
 - Composition
 - X contains Y as members



11

"Is a" ⇔ "Can do" relationship

- "Is a" relationships
 - Inheritance
 - X is derived from Y
- "Can do" relationships
 - Implementation
 - X can do (replace) the jobs of Y; but X is not Y
 - Many OO languages allows only single inheritance



12

Aggregation ⇔ Composition

- Both are “Has a” or “part-of” relationship
- Composition
 - A stronger variety of aggregation
 - A Y object may belong to only one X object
 - A Y object is expected to live and die with the X object
 - delete X → delete Y
- Aggregation
 - A Y object can be shared, i.e., a Y object may belong to several different objects

Following Larman OOAD: use of aggregation is NOT recommended

13

Example: “has a” relationship

14

Relationship Examples

• Car ⇔ Engine ?	→ Composition
• Car ⇔ Baggage ?	→ Association/Composition/Dependency
• Driver ⇔ Car ?	→ Association/Dependency
• Driver ⇔ Baggage ?	→ Association/Composition/Dependency
• Person ⇔ Cell Phone ?	→ Association/Composition/Dependency
• Human ⇔ Brain ?	→ Composition
• Fighter ⇔ Bomb ?	→ Association
• Fighter ⇔ F16	→ Inheritance
• Bomb ⇔ Explosive	→ Composition/Association/Inheritance
• Bomb ⇔ Nuclear Bomb	→ Inheritance
• Tree Node ⇔ Child of Tree Node ?	→ Composition
• Tree Node ⇔ Parent of Tree Node ?	→ Association (if needed)
• Hero ⇔ Life ? (Game)	→ Composition/Attribute
• Hero ⇔ Score ? (Game)	→ Association/Dependency
• Hero ⇔ Map ? (Game)	→ Association/Dependency

註：僅供參考，沒有一定的答案

15

Relationship Examples

• Flight 123 ⇔ Airplane ?	→ Association/Dependency
• Flight 123 ⇔ Airport ?	→ Association
• Flight 123 ⇔ Passenger ?	→ Association/Dependency
• Flight 123 ⇔ Flight Captain ?	→ Association/Dependency
• Flight 123 ⇔ Flight Attendant ?	→ Association/Dependency
• Airplane ⇔ Boeing 747 ?	→ Inheritance
• Airplane ⇔ Seat ?	→ Composition
• Airplane ⇔ Fuel ?	→ Composition/Attribute
• Passenger ⇔ Flight ?	→ Association/Dependency
• Passenger ⇔ Ticket ?	→ Association/Dependency
• Passenger ⇔ Travel Agent ?	→ Association/Dependency
• Ticket ⇔ Price ?	→ Composition/Attribute

註：僅供參考，沒有一定的答案

16

Dependency: implementation

```

class X {
    ...
    void f1(Y y) { ...; y.Foo(); }
    void f2(Y *y) { ...; y->Foo(); }
    void f3(Y &y) { ...; y.Foo(); }
    void f4() { Y y; y.Foo(); ... }
    void f5() { ...; Y::StaticFoo(); }
};
  
```

“Uses a” relationship

17

Dependency: Example

```

class Car {
public:
    void addBaggageWeight(Baggage *baggage);
private:
    double weight;
};

class Baggage {
public:
    +GetWeight() : double;
};
  
```

Parameters: in, out

Car uses Baggage

18

Association: implementation

```

class X {
    X(Y *y) : y_ptr(y) {}
    void SetY(Y *y) { y_ptr = y; }
    ...
    void f() { y_ptr->Foo(); }

    Y *y_ptr; // X keeps a pointer (reference) to Y
};

```

Two choices

SDT
Object-Oriented Programming UML and Object Relationships
Software Development and Test Laboratory • Department of Computer Science and Information Engineering • National Taipei University of Technology
19 / 36

19

Association: Example

```

class Book {
public:
    Book(string name);
    void addAuthor(Author *a);
    void listAuthors() {
        cout << "Authors of " << name
              << " are:\n";
        for(size_t i = 0; i < authors.size(); i++)
            cout << " "
                  << authors[i]->getName()
                  << endl;
    }
private:
    string name;
    vector<Author*> authors;
};

```

Book knows Author

SDT
Object-Oriented Programming UML and Object Relationships
Software Development and Test Laboratory • Department of Computer Science and Information Engineering • National Taipei University of Technology
20 / 36

20

Association: Creating an Association

How is an association created?

Example #1

```

...
Author a();
Book b(&a);
...
b.ListAuthor();
...

```

Example #2

```

...
Author a();
Book b();
...
b.addAuthor(&a);
...
b.ListAuthor();
...

```

SDT
Object-Oriented Programming UML and Object Relationships
Software Development and Test Laboratory • Department of Computer Science and Information Engineering • National Taipei University of Technology
21 / 36

21

Composition: Implementation 1/2

```

class X {
    ...
    Y a;           // 1; Composition
    Y b[10];       // 0..10; Composition
};

```

Java?

Life cycle of a and b are controlled by X

SDT
Object-Oriented Programming UML and Object Relationships
Software Development and Test Laboratory • Department of Computer Science and Information Engineering • National Taipei University of Technology
22 / 36

22

Composition: Implementation 2/2

```

class X {
    X() { y_ptr = new Y[10]; }
    ~X() { delete [] y_ptr; }
    ...
    Y *y_ptr; // 0..10; Composition
};

```

NOT an Association

SDT
Object-Oriented Programming UML and Object Relationships
Software Development and Test Laboratory • Department of Computer Science and Information Engineering • National Taipei University of Technology
23 / 36

23

Composition: Example

```

class Airplane {
public:
    void Move();
    void Draw();
private:
    Bitmap bitmap;
    int x, y;
};

```

```

void Airplane::Move() {
    x += ...;
    y += ...;
}

void Airplane::Draw() {
    bitmap.Draw(x, y);
}

```

Airplane has Bitmap

SDT
Object-Oriented Programming UML and Object Relationships
Software Development and Test Laboratory • Department of Computer Science and Information Engineering • National Taipei University of Technology
24 / 36

24

Composition: Vector/List...?

```

class X {
    ...
    vector<Y> a;
};

```

Object-Oriented Programming UML and Object Relationships

Software Development and Test Laboratory • Department of Computer Science and Information Engineering • National Taipei University of Technology

25

Aggregation

- No example here
 - Never use aggregation (Larman)
 - Use Association instead of Aggregation

Object-Oriented Programming UML and Object Relationships

Software Development and Test Laboratory • Department of Computer Science and Information Engineering • National Taipei University of Technology

26

Inheritance: Implementation

```

class Y {
    ...
};

class X : public Y {
    ...
};

```

Object-Oriented Programming UML and Object Relationships

Software Development and Test Laboratory • Department of Computer Science and Information Engineering • National Taipei University of Technology

27

Inheritance: Example

Object-Oriented Programming UML and Object Relationships

Software Development and Test Laboratory • Department of Computer Science and Information Engineering • National Taipei University of Technology

28

Implementation: C# Implementation

```

public interface Y {
    int Foo();
}

class X : Y {
    public int Foo() {
        ...
    }
}

```

Object-Oriented Programming UML and Object Relationships

Software Development and Test Laboratory • Department of Computer Science and Information Engineering • National Taipei University of Technology

29

Implementation: C++ Implementation

```

class Y {
    ...
};

class X : public Y {
    ...
};

```

Object-Oriented Programming UML and Object Relationships

Software Development and Test Laboratory • Department of Computer Science and Information Engineering • National Taipei University of Technology

30

Implementation: Example

```
classDiagram
    class IComparable {
        +CompareTo(in obj : object) : int
    }
    class MyDate
    class MyTime
    IComparable <|.. MyDate
    IComparable <|.. MyTime
```

The diagram illustrates the implementation of the `IComparable` interface. The `IComparable` interface is defined with a single method `+CompareTo(in obj : object) : int`. Two classes, `MyDate` and `MyTime`, implement this interface, as indicated by the dashed lines and hollow triangle arrows pointing from the implementation classes to the interface.

NTUST CSIE
SDIT
Laboratory

Object-Oriented Programming UML and Object Relationships

Software Development and Test Laboratory • Department of Computer Science and Information Engineering • National Taipei University of Technology

31 / 36

33

Example UML class diagram 1/2

34

Example UML class diagram 2/2

```

classDiagram
    class View {
        OnDraw()
        OnMouseDown()
        OnButtonClicked()
    }
    class simulator {
        ~simulator()
        ~simulator()
        ~simulator()
        ~simulator()
        ~simulator()
        ~simulator()
        ~simulator()
    }
    class state {
        ~state()
        ~state()
        ~state()
        ~state()
        ~state()
        ~state()
        ~state()
    }
    class device {
        ~device()
        ~device()
        ~device()
        ~device()
        ~device()
        ~device()
        ~device()
    }
    class output {
        ~output()
        ~output()
        ~output()
        ~output()
        ~output()
        ~output()
        ~output()
    }
    class ipin {
        ~ipin()
        ~ipin()
        ~ipin()
        ~ipin()
        ~ipin()
        ~ipin()
        ~ipin()
    }
    class gateAND {
        ~gateAND()
        ~gateAND()
        ~gateAND()
        ~gateAND()
        ~gateAND()
        ~gateAND()
        ~gateAND()
    }
    class gateOR {
        ~gateOR()
        ~gateOR()
        ~gateOR()
        ~gateOR()
        ~gateOR()
        ~gateOR()
        ~gateOR()
    }
    class gateNAND {
        ~gateNAND()
        ~gateNAND()
        ~gateNAND()
        ~gateNAND()
        ~gateNAND()
        ~gateNAND()
        ~gateNAND()
    }
    class gateXOR {
        ~gateXOR()
        ~gateXOR()
        ~gateXOR()
        ~gateXOR()
        ~gateXOR()
        ~gateXOR()
        ~gateXOR()
    }
    class gateNOT {
        ~gateNOT()
        ~gateNOT()
        ~gateNOT()
        ~gateNOT()
        ~gateNOT()
        ~gateNOT()
        ~gateNOT()
    }
    class fadder {
        ~fadder()
        ~fadder()
        ~fadder()
        ~fadder()
        ~fadder()
        ~fadder()
        ~fadder()
    }
    class NOR {
        ~NOR()
        ~NOR()
        ~NOR()
        ~NOR()
        ~NOR()
        ~NOR()
        ~NOR()
    }
    class truthTable {
        ~truthTable()
        ~truthTable()
        ~truthTable()
        ~truthTable()
        ~truthTable()
        ~truthTable()
        ~truthTable()
    }
    class probDelay {
        ~probDelay()
        ~probDelay()
        ~probDelay()
        ~probDelay()
        ~probDelay()
        ~probDelay()
        ~probDelay()
    }
    View --> simulator
    simulator --> state
    state --> simulator
    simulator --> device
    device --> output
    output --|> ipin
    output --|> gateAND
    output --|> gateOR
    output --|> gateNAND
    output --|> gateXOR
    output --|> gateNOT
    output --|> fadder
    output --|> NOR
    state --> layoutState
    state --> linkState
    state --|> SimStrategy
    SimStrategy --> truthTable
    SimStrategy --> probDelay
  
```

Logic circuit simulator

Object-Oriented Programming UML and Object Relationships

Software Development and Test Laboratory • Department of Computer Science and Information Engineering • National Taipei University of Technology

33 / 36

35

Exercise #1

- 練習使用UML繪圖工具繪製Class diagram
 - 請參考Lab 1

SDT Laboratory

Object Oriented Programming UML and Object Relationships

34 / 36

Software Development and Test Laboratory • Department of Computer Science and Information Engineering • National Taipei University of Technology

36

Exercise #2

- 依據原始程式繪製對應的Class Diagram
 - 請參考Lab 2

SDST CSIE
SDST
Laboratory

Object-Oriented Programming UML and Object Relationships

35 / 36

Software Development and Test Laboratory • Department of Computer Science and Information Engineering • National Taipei University of Technology

Exercise #3

- 依據需求設計Class Diagram
 - 請參考Lab 3

Object Oriented Programming UML and Object Relationships

SDTL
Laboratory

Software Development and Test Laboratory • Department of Computer Science and Information Engineering • National Taipei University of Technology

36 / 36